

SQL 是一种非过程化的查询语言，具有功能丰富、操作统一、语言简洁、易学易用等优点。但和程序设计语言相比，高度非过程化的特性造成了它的一个弱点：缺少流程控制能力，难以实现业务中的逻辑控制。数据库编程技术可以有效消除 SQL 无法实现复杂应用的问题，从而提高关系数据库管理系统支撑复杂应用的能力。

### 本章导读

本章从两个方面讲解数据库编程技术。一是扩展 SQL 自身的功能；二是在高级语言程序中使用 SQL，实现高级语言与数据库之间的交互，开发应用需求中复杂业务逻辑的技术和方法。

本章首先以实际应用任务需求为牵引，对 SQL 表达能力的限制进行说明，并阐述扩展 SQL 能力的技术途径。具体包括引入新的 SQL 子句，以增强 SQL 功能；引入新的内置函数；引入过程化 SQL (procedural language/SQL, PL/SQL) 和存储过程/存储函数的概念和技术等。

随后讲解 Java 数据库互连 (Java database connectivity, JDBC)，即在 Java 程序中如何访问数据库中的数据。最后讲解基于 MVC 框架的数据库应用开发，在实际的项目开发中，采用开发框架的方式可以提高项目开发的效率。

希望读者了解新的 SQL 子句，掌握 WITH RECURSIVE 子句的使用方法，提高 SQL 语句的可读性；掌握 PL/SQL、存储过程和存储函数等技术，以完成较为复杂的业务逻辑；能够根据实际应用场景，在第 7 章数据库设计的基础上进行应用开发，通过实践反复练习，掌握本章介绍的数据库编程方法和技术。

## 8.1 概 述

### 8.1.1 SQL 表达能力的限制

SQL 虽然功能强大，但是也有不足。本小节基于第 2 章的“学生选课”示例列举了 4 个较为

复杂的任务，这些任务直接使用基本的 SQL 语句都比较难以完成。

**任务 1** 打印“数据库系统概论”课程的所有先修课信息。

任务 1 的难点在于课程可能同时存在直接先修课和间接先修课。直接先修课的计算可以通过自身连接查询来完成，但间接先修课的计算则需要递归的查询。显然，任务 1 是一个递归的查询。其求解思路如下：

① 步骤 1。找出“数据库系统概论”课程的全部直接先修课，记为  $L[1]$ ；如果  $L[1]$  为空，则任务 1 结束。

② 步骤  $i(i \geq 2)$ 。找出集合  $L[i-1]$  中每一门课程的全部直接先修课，并计算它们的并集，记为  $L[i]$ 。

③ 迭代执行步骤  $i$ ，直到并集  $L[i]$  为空。输出  $L[1] \cup \dots \cup L[i]$ 。

在第 2 章表 2.1 所示的 Course 表中，执行步骤 1，找出“数据库系统概论”课程的直接先修课“数据结构”，即为  $L[1]$ 。执行步骤 2，找出“数据结构”课程的先修课“程序设计基础与 C 语言”，即为  $L[2]$ 。执行步骤 3，找出“程序设计基础与 C 语言”的先修课，得到  $L[3]$ 。可以发现  $L[3]$  为空，递归查询结束。根据计算结果  $L[1] \cup L[2]$ ，任务 1 的输出如表 8.1 所示。

表 8.1 任务 1 的输出结果

| Cpno  | Cname        |
|-------|--------------|
| 81002 | 数据结构         |
| 81001 | 程序设计基础与 C 语言 |

任务 1 可以使用扩展的 SQL 语句“WITH RECURSIVE 子句”完成，8.1.2 小节的例 8.2 将详细讲解。

**任务 2** 打印一周内将过生日的学生信息。

任务 2 需要数据库系统提供内置函数（本任务主要是日期函数）。其求解思路为：扫描每个学生的出生日期，例如 2000-6-12，并执行以下步骤：

① 把出生日期的年份换成当前日期所在的年份（例如 2021 年），即 2021-6-12。

② 获取当前系统的日期，例如 2021-6-9。

③ 确定过生日的日期范围 [2021-6-9, 2021-6-16]，即以当前日期为下界，当前日期后的第七天作为上界。

④ 判断出生日期是否在上述日期范围内，如果是，输出该学生信息。

8.1.2 小节的例 8.3 将讲解如何使用 SQL 的内置函数完成任务 2。

**任务 3** 给定学生学号，计算学生的平均学分绩点 GPA。

任务 3 需要用户自主设计业务处理逻辑。其求解思路为：给定学生学号，找出该学生所有选修课程的学分、成绩；根据每门课程的成绩，参照表 8.2 所示的“成绩和绩点对照表”，确定该成绩所处的范围，找出该门课程对应的绩点。

表 8.2 成绩和绩点对照表

| 编码 | 成绩下限 | 成绩上限 | 绩点 |
|----|------|------|----|
| 1  | 0    | 59   | 0  |
| 2  | 60   | 69   | 1  |
| 3  | 70   | 79   | 2  |
| 4  | 80   | 89   | 3  |
| 5  | 90   | 100  | 4  |

以学号为“20180001”的学生为例，该同学共选修了三门课程，如表 8.3 所示。

表 8.3 学号为“20180001”的学生的选修课程

| 学号<br>Sno | 课程号<br>Cno | 成绩<br>Grade | 开课学期<br>Semester | 教学班<br>Teachingclass |
|-----------|------------|-------------|------------------|----------------------|
| 20180001  | 81001      | 85          | 20192            | 81001-01             |
| 20180001  | 81002      | 96          | 20201            | 81002-01             |
| 20180001  | 81003      | 87          | 20202            | 81003-01             |

课程号为“81001”的课程考试成绩为 85 分，参照表 8.2，该成绩对应的成绩范围为[80, 89]，绩点为 3；类似地，课程号为“81002”的课程考试成绩对应的绩点为 4，课程号为“81003”的课程考试成绩对应的绩点为 3。从 Course 表可知，这三门课程的学分都是 4。根据平均学分绩点的计算公式，即  $GPA = (\text{每门课程的学分} * \text{对应的绩点}) \text{的总和} / \text{学分的总和}$ ，得到该同学的 GPA 为 3.33。因此，上述查询结果为

GPA

3.33

8.2.5 小节的例 8.5 将介绍如何使用存储过程来完成任务 3。

**任务 4** 学生通过交互界面提交对某一位任课教师的教学评价意见，教师浏览这些评价意见并提供反馈信息。

任务 4 需要建立交互功能。一方面，学生需要找到指定的教学班和授课教师，建立如图 8.1 所示的交互界面并输入课程评价。另一方面，还要建立如图 8.2 所示的交互界面，教师浏览教学班学生的评价意见，并针对每条评价逐一做出回复。

8.3.3 小节的例 8.13 将讲解如何基于 JDBC 实现任务 4。

类似任务 1~任务 4 的应用还有很多，直接使用 SQL 实现都比较困难。那么如何扩展 SQL 的表达能力呢？主要有两大类途径：扩展 SQL 的功能和通过高级语言实现复杂应用。下面详

细介绍这两种途径。

| 课程评教     |                                                                                               |
|----------|-----------------------------------------------------------------------------------------------|
| <i>!</i> | 信息                                                                                            |
| 教学班      | 81001-01                                                                                      |
| 课程       | 程序设计基础与C语言                                                                                    |
| 任课老师     | 姜山                                                                                            |
| 课程评价     | <div> <p>总体上老师讲得很好。在数据库设计部分, 建议增加更丰富的应用实例</p> <p>填写课程评价</p> <p>填写完毕后点击添加</p> <p>添加</p> </div> |

图 8.1 学生输入并提交课程评价

| 查看教学班: 81001-01的学生评教 |                                  |                         |
|----------------------|----------------------------------|-------------------------|
| 学号                   | 评论                               | 操作                      |
| 20180001             | 感谢老师                             | 感谢认可                    |
| 20180002             | 感谢老师                             | 感谢认可                    |
| 20180003             | 总体上老师讲得很好。在数据库设计部分, 建议增加更丰富的应用实例 | <input type="text"/> 回复 |

教师输入对学生评价的反馈 输入完毕后 点击回复

图 8.2 教师提交对学生评价的反馈

### 8.1.2 扩展 SQL 的功能

SQL 标准从发布以来随数据库技术的发展而不断丰富。通过扩展 SQL 可使其表达功能更为强大, 进而支持更为复杂的应用。SQL 扩展的方式主要包括引入新的 SQL 子句、引入新的内置函数, 以及引入 PL/SQL 与存储过程/存储函数。

#### 1. 引入新的 SQL 子句

以任务 1 为例, SQL 标准引入了 WITH RECURSIVE 子句, 可执行递归查询。

类似于 WITH RECURSIVE 子句的 SQL 扩展还有很多, 例如面向联机分析处理的窗口子句, 面向空间数据管理、文档数据管理的 SQL 扩展等。

在介绍 WITH RECURSIVE 子句之前, 先了解 WITH 子句的用法。

##### (1) WITH 子句

WITH 子句用来创建一个命名的临时结果集, 该结果集仅在 SQL 语句(例如 SELECT,

INSERT或DELETE)执行时有效,不长期存储。WITH子句的引入,可以提高SQL语句的性能和可读性。

WITH子句的语法格式如下:

```
WITH RS1[(<目标列>,<目标列>)]AS(SELECT 语句1)[,
    RS2[(<目标列>,<目标列>)]AS(SELECT 语句2),...]
SQL 语句;
```

RS1为临时结果集的命名,RS1对应SELECT语句1的执行结果,SELECT语句1中的目标列与RS1中的目标列必须保持一致。

RS2为临时结果集的命名,RS2对应SELECT语句2的执行结果,SELECT语句2中的目标列与RS2中的目标列必须保持一致。SQL语句执行与RS1,RS2...相关的查询。

[例8.1]求81001-01和81001-02两个教学班之间学生选课平均成绩的差异。

```
WITH RS1(grade) AS
    (SELECT AVG(grade) FROM SC WHERE Teachingclass = '81001-01'),
    RS2(grade) AS
    (SELECT AVG(grade) FROM SC WHERE Teachingclass = '81001-02')
SELECT RS1.grade-RS2.grade FROM RS1, RS2
```

## (2) WITH RECURSIVE 子句

WITH RECURSIVE子句是WITH子句的一种特殊情况,用来查找具有层次结构的数据。WITH RECURSIVE子句由两部分组成:第一部分是种子查询,用来初始化临时结果集,即第一层数据,假设记为 $L[1]$ ;第二部分是递归查询,该查询以 $L[1]$ 为基础,查找第二层数据 $L[2]$ ,然后以 $L[2]$ 为基础,查找第三层数据 $L[3]$ ,以此类推。递归查询需要设定每一层次数据的查找方式,比如查找第二层的数据时,需要用第一层数据中某个属性(或属性组)和关系表中的另外一个属性(或属性组)进行匹配,按照这个条件查找出来的数据就是第二层数据,同理查找第三层、第四层数据等。

WITH RECURSIVE子句的语法格式如下:

```
WITH RECURSIVE RS AS
(
    SEED QUERY           /* 初始化查询的临时结果集,记为 L[1] */
    UNION [ALL]          /* 是否需要保留重复记录,加 ALL 为保留 */
    RECURSIVE QUERY      /* 执行递归查询,得到全部临时结果集,即 L[2] ∪ ... ∪ L[i] */
)
SQL 语句 /* 执行与 RS 相关的查询 */
```

[例 8.2] 打印“数据库系统概论”课程的所有先修课信息。

WITH RECURSIVE RS AS

( SELECT Cjno FROM Course WHERE Cname = '数据库系统概论'

/\* 初始化 RS, 假设结果集为 L[1], 即“数据库系统概论”的所有直接先修课 \*/

UNION

SELECT Course.Cjno FROM Course, RS WHERE RS.Cjno = Course.Cno

) /\* 递归查询第 i 层 ( $i \geq 1$ ) 的数据, 即第 i-1 层数据的直接先修课课程号, 并更新 RS \*/

SELECT Cno, Cname FROM Course WHERE Cno IN (SELECT Cjno FROM RS);

/\* 根据 RS 中记录的所有先修课程号, 通过查找课程表, 输出课程号与课程名 \*/

第2章图2.1所示的 Course 表中, 课程之间的先修课关系具有层次结构, 如图8.3所示。种子查询返回的是以“81003 数据库系统概论”课程为根的第一层数据, 即“数据库系统概论”课程的直接先修课 **Cjno=81002**, 这时 RS 被初始化为 81002。执行递归查询, RS 表与 Course 表做自然连接, 查找课程 Cno=81002 的先修课, 返回其先修课 **Cno=81001**, 这是第二层数据。用 RS 表中的第二层数据与 Course 表做自然连接, 查找课程 Cno=81001 的先修课, 这是第三层数据。因为 Cno=81001 的课程不存在直接先修课, 因此第三层数据为空。这时, 递归查询结束。“数据库系统概论”课程 81003 所有的先修课有 81002、81001。



图 8.3 课程之间的先修课关系具有明显的分层结构

**注意：**使用 WITH RECURSIVE 子句查找所有先修课课程信息，同样适用于一门课程存在多门先修课的场景。

## 2. 引入新的内置函数

关系数据库管理系统提供了丰富的内置函数，方便用户的操作。



扩展阅读：几个数据库产品的部分常用内置函数  
(1)

常用的基本内置函数可以分为数学函数(如绝对值函数等)、聚合函数(如求和、求平均函数等)、字符串函数(如求字符串长度、求子串函数等)、日期和时间函数(如返回当前日期函数等)、格式化函数(如字符串转 IP 地址函数等)、控制流函数(如逻辑判断函数等)、加密函数(如使用密钥对字符串加密函数等)、系统信息函数(如返回当前数据库名、服务器版本函数)等。本书整理了几个数据库产品的部分常用内置函数,读者可以阅读二维码中的相关内容。



扩展阅读:几个数据库产品的部分常用内置函数  
(2)

[例 8.3]打印一周内将过生日的学生信息。

```
SELECT Sno, Sname, Ssex, Sbirthdate, Smajor
FROM Student
WHERE to_date(to_char(current_date,'yyyy') || '-' || to_char(Sbirthdate,'mm-dd'))
BETWEEN current_date AND current_date + interval '7' day
```

在 WHERE 语句中使用了以下内置函数:

- ① 内置函数 `current_date` 返回当前的系统日期,例如 2021-6-9。
- ② 内置函数 `to_char(current_date, 'yyyy')` 返回当前系统日期的年份,例如 2021; `to_char(Sbirthdate, 'mm-dd')` 返回学生出生日期中具体的月份和日期,例如 6-9。
- ③ `to_date(to_char(current_date, 'yyyy') || '-' || to_char(Sbirthdate, 'mm-dd'))` 表示把当前年份与出生日期用 '-' 连在一起。符号 "||" 用于把其左右两边的字符串连在一起。
- ④ 内置函数 `to_date()` 的作用是将字符类型按一定格式转化为日期类型。
- ⑤ `current_date + interval '7' day` 是对当前的日期调整后的日期。参数 `interval` 是年(yyyy)、季度(q)、月(m)、日(d)、时(h)等粒度的时间单位。

例如, `current_date + interval '7' day` 获得当前日期之后第七天的日期,即返回 2021-6-16。

通过执行此 WHERE 语句,判断学生表中每位学生转换后的出生日期是否在 [2021-6-9, 2021-6-16] 区间内,如果是,打印该学生的信息。

### 3. 引入 PL/SQL 与存储过程/存储函数

内置函数是关系数据库管理系统默认创建的。此外,还可以通过安装数据库的扩展插件添加该插件自带的内置函数。但是,在实际应用中仅仅使用关系数据库管理系统默认的或扩展插件自带的内置函数还是不够的。因此,在关系数据库管理系统中引入了 PL/SQL、存储过程/存储函数等方法,使得用户可以自定义程序逻辑,开发完成业务逻辑复杂的应用系统。

例如,在 8.1.1 小节的任务 3 中,用户需要自定义计算平均学分绩点的函数。在该函数中引入逻辑判断和循环控制,逻辑判断用于获取每门课程成绩对应的绩点,循环控制用于计算课程总学分和总学分绩点,最终计算得到平均学分绩点。我们将在 8.2.5 小节的例 8.5 详细给出求解方法。



### 8.1.3 通过高级语言实现复杂应用

对于类似于 8.1.1 小节任务 4 的应用逻辑和功能需求,用户和数据库之间需要通过界面进行交互,这时可以通过高级语言(如 Java、C++、Python 等)访问数据库来解决。为此需要在关系数据库管理系统和应用系统之间建立交互接口,相互传递数据和控制信息,实现两类系统之间的互动。

作为各自独立的系统,关系数据库管理系统和应用系统之间的互操作是计算机软件发展中非常重要的一个话题。随着技术的进步和观念的更新,其交互方式也一直在丰富和发展,主要有以下几种方式。

#### 1. 通过动态链接库调用的方式

在大型软件系统构建过程中,为了提高软件生产率,提出了“模块化”的概念,也就是将大型软件分解为一些小的模块,每个模块可以组织成“子程序”(或者“函数”),之后通过在一个程序中“调用”(CALL)另一个子程序来实现程序与子程序之间的交互。在这样的思路下,关系数据库管理系统的功能被包装成一个子程序,由应用程序通过动态链接库调用来获得数据管理(包括查询、管理等)的功能。

#### 2. 基于嵌入式 SQL 的方式

嵌入式 SQL 是将 SQL 语句嵌入某程序设计语言,被嵌入的程序设计语言称为宿主语言,简称主语言。

对嵌入式 SQL,关系数据库管理系统一般采用预编译方法处理,即由其预编译程序对源程序进行扫描,识别出嵌入式 SQL 语句并将其转换成主语言的调用语句(对库函数的调用),再由主语言的编译程序将其编译成目标码。嵌入式 SQL 编程的好处在于把用户从复杂的函数调用中解放出来,用户只需要在主语言中使用 SQL 语句代替复杂的函数调用即可,从而较大幅度地简化了开发工作量。

将 SQL 嵌入高级语言中混合编程,SQL 语句负责操纵数据库,高级语言语句负责控制逻辑流程,这时程序中会含有两种不同计算模型的语句,它们之间应如何通信呢?

数据库工作单元与源程序工作单元之间的通信主要包括 SQL 通信区、主变量、游标等方式。

##### (1) SQL 通信区

SQL 通信区(SQL communication area, SQLCA)主要实现向主语言传递 SQL 语句的执行状态信息,使主语言能够据此信息控制程序流程。

SQL 语句执行后,系统要反馈给应用程序若干控制信息,主要包括描述数据库系统当前工作状态和运行环境的各种数据。这些信息将被送到 SQLCA 中,应用程序从 SQLCA 中取出这些状态信息,据此决定接下来执行的语句。

SQLCA 由关系数据库管理系统预先定义好,根据高级语言的不同有所不同。SQLCA 将作为高级语言变量定义的一部分而存在。



例如, SQLCA 中有一个变量 SQLCODE, 用来存放每次执行 SQL 语句后返回的代码。应用程序每执行完一条 SQL 语句之后都应测试一下 SQLCODE 值, 以了解该 SQL 语句是否执行成功。如果 SQL 语句执行不成功, 则在 SQLCODE 中存放的就是错误代码, 程序员可以根据错误代码查找问题, 应用程序也可以根据错误代码执行相应的处理。

## (2) 主变量

主语言主要用主变量(host variable)向 SQL 语句提供参数。

嵌入式 SQL 语句中可以使用主语言的程序变量来输入或输出数据, 这些程序变量简称为**主变量**。主变量根据其作用的不同可分为输入主变量和输出主变量。**输入主变量**由应用程序对其赋值, SQL 语句引用;**输出主变量**由 SQL 语句对其赋值或设置状态信息, 返回给应用程序。

一个主变量可以附带一个任选的指示变量(indicator variable)。**指示变量**是一个整型变量, 用来“指示”所指主变量的值或条件。指示变量可以指示输入主变量是否为空值, 可以检测输出主变量是否为空值、值是否被截断等。

所有主变量和指示变量必须在 SQL 语句“BEGIN DECLARE SECTION”与“END DECLARE SECTION”之间进行说明。说明之后, 主变量可以在 SQL 语句中任何一个能够使用表达式的地方出现, 为了与数据库对象名(表名、视图名、列名等)区别, SQL 语句中的主变量名和指示变量名前要加冒号(:)作为标志。

## (3) 游标

SQL 是面向集合的, 一条 SQL 语句可以产生或处理多条记录; 而高级语言是面向记录的, 一次只能处理一条记录。所以在 SQL 语句向应用程序输出数据时, 需要有一个内存区域来缓存数据, 这个区域就称为游标。用游标来协调这两种不同的处理方式。

**游标**是关系数据库管理系统为应用程序开设的一个数据缓冲区, 存放 SQL 语句的执行结果, 每个游标区都有一个名字和一个指针。应用程序可以通过游标指针逐一获取记录并进行处理。

## 3. 基于 ODBC/JDBC 的中间件方式

目前广泛使用的关系数据库管理系统有多种, 尽管这些系统都遵循 SQL 标准, 但是不同的系统有许多差异。因此, 在某个关系数据库管理系统下编写的应用程序并不能在另一个关系数据库管理系统下运行, 适应性和可移植性较差。例如, 运行在 Oracle 上的应用系统要在 KingbaseES 上运行, 就必须进行修改移植。这种修改移植比较烦琐, 开发人员必须清楚地了解不同关系数据库管理系统的区别, 细心地一一进行修改、测试。更重要的是, 许多应用程序需要共享多个部门的数据资源, 访问不同的关系数据库管理系统甚至文件系统。为此, 人们开始研究和开发连接不同关系数据库管理系统的方法、技术和软件, 使数据库系统“开放”, 能够实现数据库互连。

微软公司提出了 Windows 开放服务体系结构(Windows open services architecture, WOSA)的概念, 其核心思想是将微软平台上所有独立的软件系统都作为“设备”统一看待并标准化(如同打印机这类硬件设备)。设备的生产厂商按照标准提供驱动程序, 这样微软平台上的应用程序

就可以按照统一的方式访问不同生产厂商的设备了。应用程序使用某个具体设备之前只需安装该设备的驱动程序,并按照标准的接口方式使用即可,从而最大限度地规范了应用程序的开发。具体到数据库的场景,就是**开放数据库互连**(open database connectivity, ODBC)。

ODBC 是微软公司 WOSA 中有关数据库的一个组件,它建立了一组规范,并提供一组数据库应用编程接口(database application programming interface, DBAPI)。这样,无论使用什么数据库,都采用同样的一组编程接口来访问数据库,规范了应用程序的开发,提高了应用程序的可移植性。

数据库系统需要各自实现这组编程接口,形成该数据库的驱动程序。利用 ODBC 访问数据库之前需要安装数据库驱动程序, WOSA 中有一个数据库驱动程序管理器,负责管理各种数据库的驱动程序。

利用 ODBC 访问数据库的流程如下:

- ① 环境准备。配置数据源,即某个数据库,包含数据库名称,访问地址,用户名,口令等信息。需要用驱动程序管理器工具来完成。
- ② 设置环境。利用编程接口中的环境设置语句进行设置。
- ③ 进行数据库连接。
- ④ 完成相关操作。执行 SQL 语句,完成对数据库的相关操作、对返回结果集进行计算等。
- ⑤ 关闭数据库连接,释放占用的资源。

类似地,在微软之外的平台上,也有其他的一些机制,例如 JDBC(适用于 Java 环境)等。由于 Java 语言使用的普遍性,本书将在 8.3 节详细介绍如何使用 JDBC 访问数据库中的数据。

软件的体系结构在不断发展变化,访问数据库的方式也会随之发生变化,该过程不会终止。期待有更便捷高效的应用程序与数据库之间的交互方式的出现。

## 8.2 过程化 SQL

SQL 99 标准支持存储过程和函数的概念,商用关系数据库管理系统大多提供过程化的 SQL,如 Oracle 的 PL/SQL、Microsoft SQL Server 的 Transact-SQL、IBM DB2 的 SQL PL、Kingbase 的 PL/SQL 等。本节主要介绍过程化 SQL 的块结构、变量和常量的定义、流程控制、游标的定义与使用、存储过程及存储函数等内容。

### 8.2.1 过程化 SQL 的块结构

基本 SQL 是高度非过程化的语言,而过程化 SQL 是对基本 SQL 的扩展,在其基础上增加了一些描述过程控制的语句。

**过程化 SQL 程序的基本结构是块**,每个块可以包含定义部分和执行部分,块与块之间可以相互嵌套,每个块完成一个逻辑操作。图 8.4 是过程化 SQL 块的基本结构。